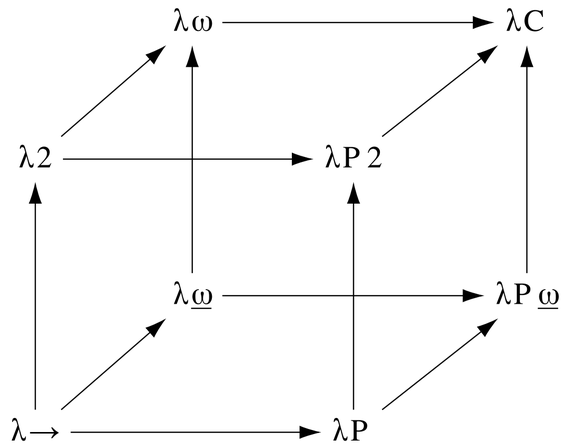
Theory of Programming Languages

10. Type Operator

Kihong Heo

KAIST

Motivation



- Type-level generic programming: independent of specific types
- Core idea: types depend on other types
- Type operator: functions at the level of types
  - E.g., type constructor, functor (OCaml), generic (Java)
- Higher-order polymorphic lambda calculus
  - $\lambda\omega$ or System F_ω
- Foundation of many mainstream programming languages like OCaml

Example

- Pair in System F

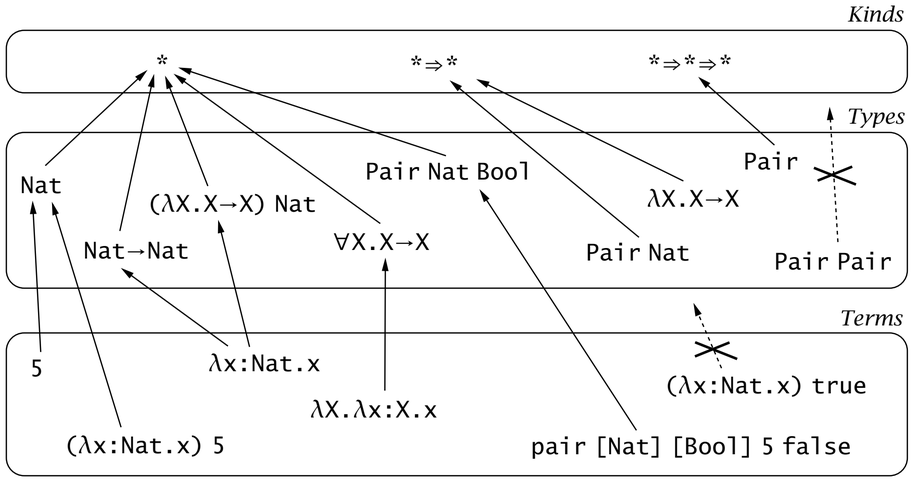
$$A \times B = \forall \alpha.(A \to B \to \alpha) \to \alpha$$

$$\texttt{pair} = \lambda a{:}A.\lambda b{:}B.\Lambda\alpha.\lambda f{:}A \to B \to \alpha.f\ a\ b$$
$$\texttt{fst} = \lambda p{:}A \times B.p\ A\ (\lambda a{:}A.\lambda b{:}B.a)$$
$$\texttt{snd} = \lambda p{:}A \times B.p\ B\ (\lambda a{:}A.\lambda b{:}B.b)$$

- What machinery do we need to generalize pairs?
  - “type” of types: kind
  - “function” of types: type operator
- Answer: type operator

$$\text{Pair} = \lambda A{:}*.\lambda B{:}*.\forall X.(A \to B \to X) \to X$$

Kind



- Types of types
  - $*$: kind of proper types
  - $* \Rightarrow *$: kind of type operators
  - $* \Rightarrow * \Rightarrow *$: kind of funs from proper types to type operators
  - $(* \Rightarrow *) \Rightarrow *$: kind of functions from type operators to proper types
  - …
- System of kinds: type-level simply typed lambda calculus

*Figure from Types and Programming Languages

Syntax & Dynamic Semantics

- Syntax, evaluation context, values, and kinds
  - Kinds: proper types or type operators

$$
\begin{array}{rcl}
E & \to & x \mid \lambda x{:}\tau.E \mid E\ E \\
 & \mid & \Lambda X{:}\kappa.E \mid E\ \tau
\end{array}
\qquad
\begin{array}{rcl}
K & \to & [\,] \mid K\ E \mid v\ K \\
 & \mid & K\ \tau
\end{array}
$$

$$
\begin{array}{rcl}
\tau & \to & X \\
 & \mid & \tau \to \tau \\
 & \mid & \forall X{:}\kappa.\ \tau \\
 & \mid & \lambda X{:}\kappa.\ \tau \\
 & \mid & \tau\ \tau
\end{array}
\qquad
\begin{array}{rcl}
v & \to & \lambda x{:}\tau.E \mid \Lambda X{:}\kappa.E \\
 & & \\
\kappa & \to & * \\
 & \mid & \kappa \Rightarrow \kappa
\end{array}
$$

- Dynamic Semantics

$$
\frac{E \to E'}{K[E] \to K[E']}
\qquad
\frac{}{(\lambda x{:}\tau.E)\ v \to E\{v/x\}}
\qquad
\frac{}{(\Lambda X{:}\kappa.E)\ \tau \to E\{\tau/X\}}
$$

Static Semantics (1): Kinding Rules

$$\text{Kind env} \quad \Delta \quad \in \quad \text{TypeVar} \to \text{Kind}$$

$$\frac{\Delta(X) = \kappa}{\Delta \vdash X : \kappa} \qquad \frac{\Delta, X : \kappa_1 \vdash \tau_2 : \kappa_2}{\Delta \vdash \lambda X : \kappa_1 . \tau_2 \ : \ \kappa_1 \Rightarrow \kappa_2} \qquad \frac{\Delta \vdash \tau_1 : \kappa_1 \Rightarrow \kappa_2 \qquad \Delta \vdash \tau_2 : \kappa_1}{\Delta \vdash \tau_1 \ \tau_2 : \kappa_2}$$

$$\frac{\Delta \vdash \tau_1 : * \qquad \Delta \vdash \tau_2 : *}{\Delta \vdash \tau_1 \to \tau_2 : *} \qquad \frac{\Delta, X : \kappa \vdash \tau : *}{\Delta \vdash \forall X : \kappa. \ \tau : *}$$

Static Semantics (2): Typing Rules

- Typeable terms always have kindable types: $\Delta, \Gamma \vdash E : \tau$ then $\Delta \vdash \tau : *$

$$\text{Type env} \quad \Gamma \quad \in \quad \mathsf{Var} \to \mathsf{Type}$$

$$\frac{\Gamma(x) = \tau}{\Delta, \Gamma \vdash x : \tau} \qquad \frac{\Delta \vdash \tau_1 : * \qquad \Delta, \Gamma + x : \tau_1 \vdash E : \tau_2}{\Delta, \Gamma \vdash \lambda x{:}\tau_1.E : \tau_1 \to \tau_2} \qquad \frac{\Delta, \Gamma \vdash E_1 : \tau_1 \to \tau_2 \qquad \Delta, \Gamma \vdash E_2 : \tau_1}{\Delta, \Gamma \vdash E_1\ E_2 : \tau_2}$$

$$\frac{\Delta + X{:}\kappa, \Gamma \vdash E : \tau \qquad X \notin \mathsf{Dom}(\Delta)}{\Delta, \Gamma \vdash \Lambda X{:}\kappa.\ E : \forall X{:}\kappa.\ \tau} \qquad \frac{\Delta \vdash \tau{:}\kappa \qquad \Delta, \Gamma \vdash E : \forall X{:}\kappa.\ \tau'}{\Delta, \Gamma \vdash E\ \tau : \tau'\{\tau/X\}}$$

$$\frac{\Delta, \Gamma \vdash E : \sigma \qquad \sigma \equiv \tau \qquad \Delta \vdash \tau : *}{\Delta, \Gamma \vdash E : \tau}$$

Static Semantics (3): Type Equivalence

- Definitional equivalence rules (정의에 따른 동치 규칙)

$$\frac{}{\tau \equiv \tau} \qquad \frac{\tau \equiv \sigma}{\sigma \equiv \tau} \qquad \frac{\tau_1 \equiv \tau_2 \qquad \tau_2 \equiv \tau_3}{\tau_1 \equiv \tau_3}$$

$$\frac{\tau_1 \equiv \sigma_1 \qquad \tau_2 \equiv \sigma_2}{\tau_1 \to \tau_2 \equiv \sigma_1 \to \sigma_2} \qquad \frac{\tau \equiv \sigma}{\lambda X:\kappa.\tau \equiv \lambda X:\kappa.\sigma} \qquad \frac{\tau_1 \equiv \sigma_1 \qquad \tau_2 \equiv \sigma_2}{\tau_1\ \tau_2 \equiv \sigma_1\ \sigma_2}$$

$$\frac{}{(\lambda X:\kappa.\tau_1)\tau_2 \equiv \tau_1\{\tau_2/X\}}$$

Example

- Pair in System F

$$A \times B = \forall X.(A \to B \to X) \to X$$

$$\begin{aligned}
\texttt{pair} &= \lambda a{:}A.\lambda b{:}B.\Lambda X.\lambda f{:}A \to B \to X.f\ a\ b \\
\texttt{fst} &= \lambda p{:}A \times B.p\ A\ (\lambda a{:}A.\lambda b{:}B.a) \\
\texttt{snd} &= \lambda p{:}A \times B.p\ B\ (\lambda a{:}A.\lambda b{:}B.b)
\end{aligned}$$

- Pair in System Fw

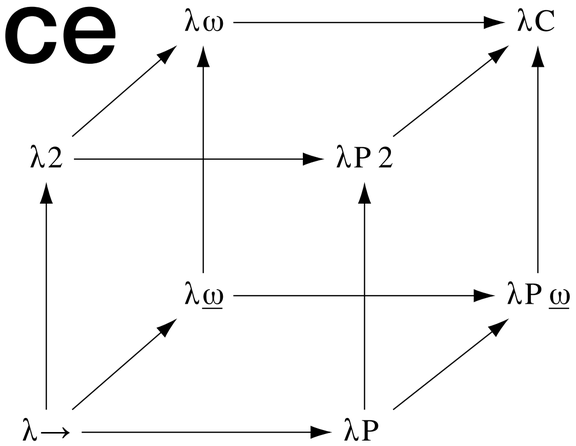
$$\mathsf{Pair} = \lambda A{:}*.\lambda B{:}*.\forall X{:}*.(A \to B \to X) \to X$$

$$\begin{aligned}
\texttt{pair} &= \Lambda A{:}*.\Lambda B{:}*.\lambda a{:}A.\lambda b{:}B.\Lambda X{:}*.\lambda p{:}A \to B \to X.p\ a\ b \\
\texttt{fst} &= \Lambda A{:}*.\Lambda B{:}*.\lambda p{:}\mathsf{Pair}\ A\ B.p\ A\ (\lambda a{:}A.\lambda b{:}B.a) \\
\texttt{snd} &= \Lambda A{:}*.\Lambda B{:}*.\lambda p{:}\mathsf{Pair}\ A\ B.p\ B\ (\lambda a{:}A.\lambda b{:}B.b)
\end{aligned}$$

Exercise

- `pair` int bool 3 `true`
- `fst` int bool (`pair` int bool 3 `true`)

Curry-Howard Correspondence



- $\lambda\omega$: System F + type operators
  - Type = proposition, Type operator = proposition operator
- Corresponding to higher-order propositional logic
- Recall
  - $P \wedge Q \to P$: propositional (zeroth-order)
  - $\forall x \in \mathbb{N} \,.\, Even(2x)$: predicate (first-order)
  - $\forall P \in \mathbb{N} \to \text{Prop} \,.\, (P(0) \wedge \forall k \,.\, (P(k) \to P(k+1)) \to \forall n \,.\, P(n))$: second-order
  - $\forall F \in (\text{Prop} \to \text{Prop}) \to \text{Prop} \,.\, F(P) \to F(P)$: third-order (higher-order)

Summary

- System F_ω consists of
  - terms depend on terms: STLC
  - terms depend on types: System F
  - types depend on types: type operator
- Curry-Howard correspondence: higher-order propositional logic